



Pré-processamento de Dados em Machine Learning: Entendendo o Problema, Analisando e Processando os Dados

Aula 3

Antes de qualquer modelo, o sucesso de um projeto de Machine Learning é definido pela qualidade de suas bases. A compreensão do problema e a Análise Exploratória de Dados (EDA) nos permitem **diagnosticar a saúde dos dados**. É este diagnóstico que guiará cada passo subsequente, desde a limpeza até a engenharia de atributos, garantindo a solidez de nosso projeto

Professor: Dr. Rodrigo Ramos Silva

2026/2

Objetivos de Aprendizagem

- 1. Higiene mínima determinística**
- 2. Split inicial (teste sagrado)**
- 3. Checagem de esquema (regras fixas)**
- 4. EDA no treino**
- 5. Pipeline de pré-processamento (fit no treino; aplicar em val/teste)**
- 6. Criação/**
- 7. Extração de Features (visão)**
- 8. Seleção de Features (visão)**

A Importância do Entendimento do Problema

Antes de qualquer análise de dados, precisamos compreender completamente o problema que estamos tentando resolver.

1º

Passo Essencial

O entendimento do problema é a etapa mais importante no design de um sistema de Machine Learning.

70%

Taxa de Sucesso

Uma definição precisa do problema aumenta significativamente a probabilidade de construir um produto eficiente.

15min

Tempo Valioso

Mesmo poucos minutos de observação atenta podem gerar insights que economizam horas de trabalho.

Uma compreensão profunda do problema fornece a base para todas as etapas técnicas subsequentes, desde a coleta de dados até a implantação do modelo.

Definição do Problema: Perguntas Fundamentais

Durante a fase de definição do problema, devemos investigar profundamente o "o quê" e o "porquê" da questão que estamos abordando.



Por que precisamos construir essa solução?

Identifique as motivações reais por trás do projeto e se uma solução de ML é realmente necessária.



Qual problema específico ela resolve?

Articule claramente o problema e defina seus limites para evitar soluções genéricas ou mal direcionadas.



Por que o problema ocorre?

Compreenda as causas subjacentes do problema para desenvolver uma solução mais efetiva.



Quais são as alternativas conhecidas?

Investigue soluções existentes e por que elas podem não ser adequadas para o contexto específico.



Quais são as limitações do projeto?

Identifique restrições de métricas, latência, quantidade de dados de treinamento e recursos disponíveis.

O caminho seguro de um projeto de ML

1. Entender o problema

O que queremos prever, classificar ou estimar?

2. Definir alvo e critério de sucesso

Qual variável queremos prever? Como saber se a solução é boa?

3. Separar os dados corretamente

Treino, validação e teste.

4. Conhecer os dados de treino

Estrutura, qualidade, padrões, faltantes, outliers, distribuição.

5. Decidir o pré-processamento

Imputação, codificação, escala, tratamento de variáveis.

6. Criar e selecionar atributos

Quais informações ajudam mais o modelo?

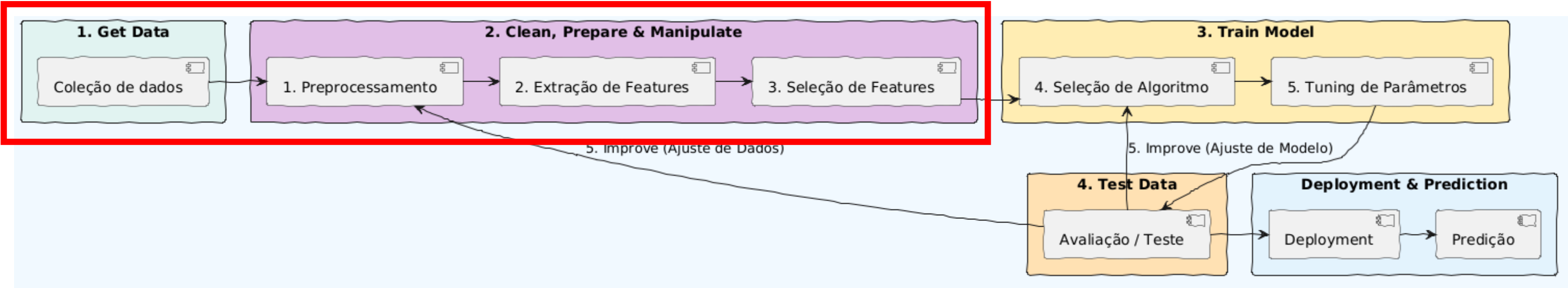
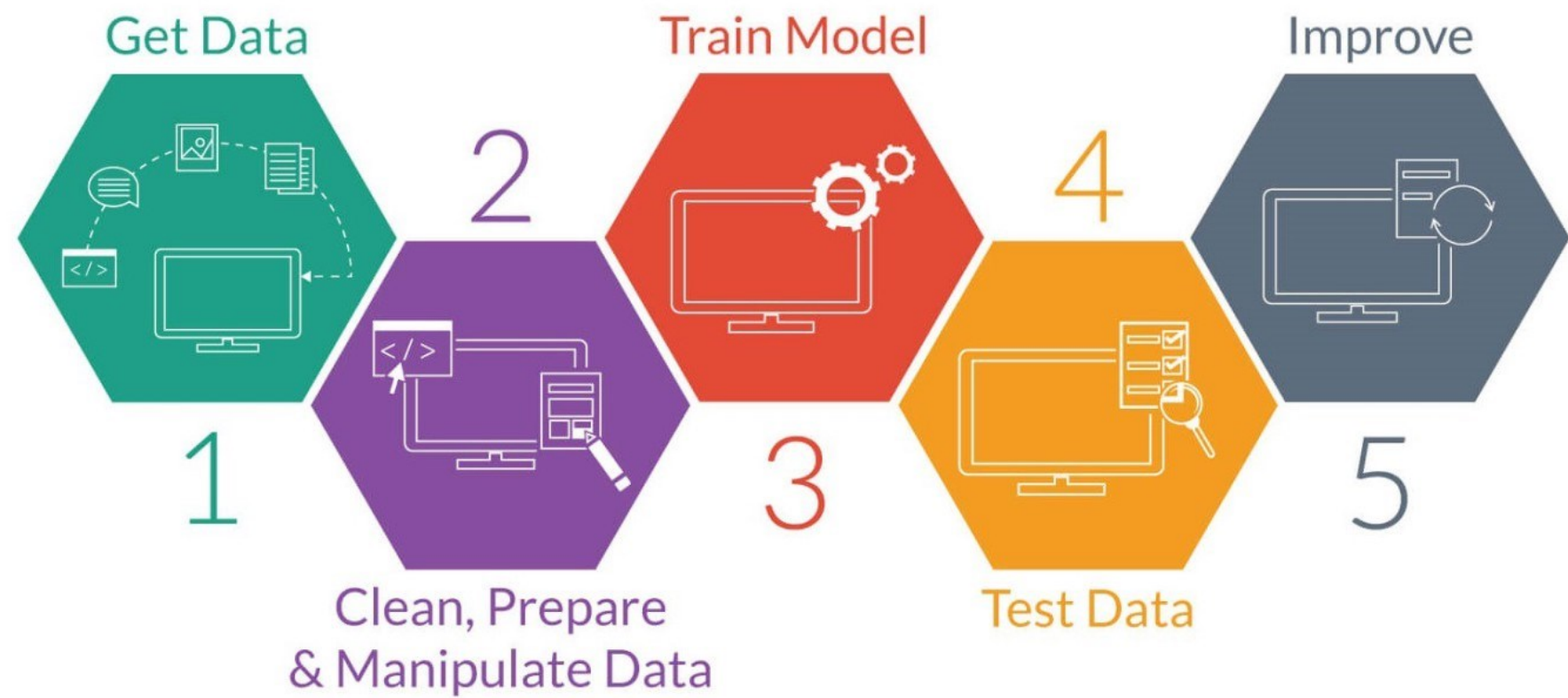
7. Treinar e validar

Ajustar no treino e validar sem contaminar o teste.

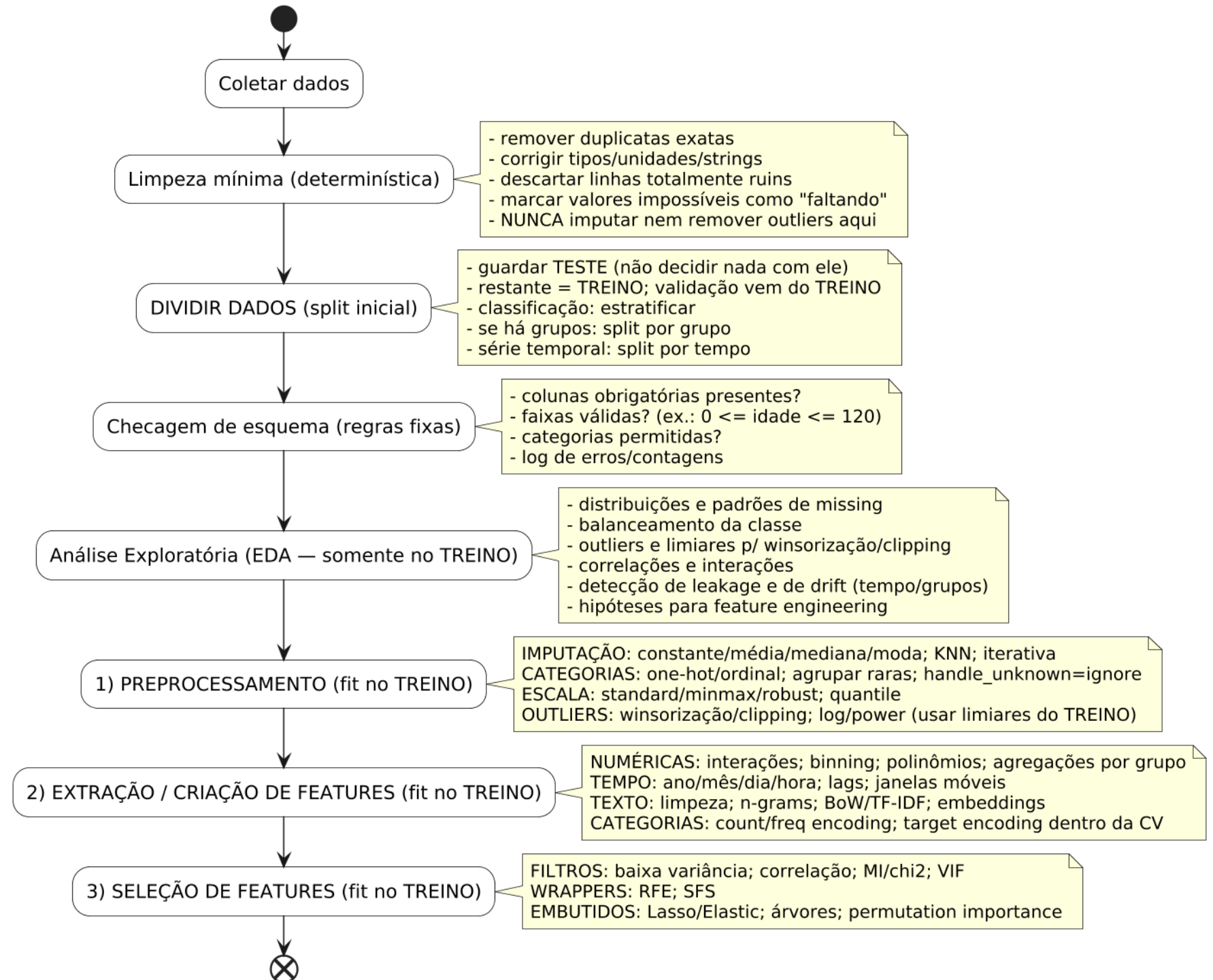
8. Testar a solução final

Usar o teste apenas no final.

Etapas de Desenvolvimento Machine Learning



Etapas de Desenvolvimento Machine Learning



1. Limpeza e Divisão (O Início)

- Determinística:** Algo que segue uma regra fixa e lógica (ex: "se o valor é negativo e deveria ser positivo, multiplique por -1"). Não depende de estatística complexa.
- Imputar:** Preencher valores que estão faltando (os famosos NaN ou null) com alguma estimativa.
- Split Inicial:** Dividir seus dados em conjuntos de **Treino** (para o modelo aprender) e **Teste** (para ver se ele aprendeu mesmo ou só decorou).
- Estratificar:** Garantir que a proporção das classes (ex: 10% de fraudes e 90% de vendas normais) seja a mesma tanto no treino quanto no teste.

2. EDA (Análise Exploratória)

- Outliers:** Valores "fora da curva" (ex: uma pessoa com 200 anos de idade).
- Winsorização/Clipping:** Técnicas para "podar" os outliers. Em vez de deletar o dado, você o limita a um valor máximo ou mínimo.
- Leakage (Vazamento):** Quando informações do futuro ou do conjunto de teste "vazam" para o treino, fazendo o modelo parecer muito melhor do que realmente é.
- Drift:** Quando os dados mudam com o passar do tempo (ex: comportamento de compra antes e depois de uma pandemia).

3. Pré-processamento e Escala

- One-Hot Encoding:** Transformar categorias (ex: cores "Vermelho", "Azul") em colunas de 0 e 1.
- Standard/MinMax/Robust:** Formas de colocar os números na mesma escala (ex: transformar salários e idades para que ambos fiquem entre 0 e 1).
- Log/Power:** Transformações matemáticas para fazer dados muito "espalhados" ficarem mais próximos de uma distribuição normal (o famoso desenho do sino).

4. Extração e Criação (Feature Engineering)

- Binning:** Agrupar valores numéricos em "baldes" (ex: transformar idades de 0 a 100 em "Criança", "Adulto", "Idoso").
- N-grams / BoW / TF-IDF:** Formas de transformar textos em números para que o computador entenda.
- Embeddings:** Representações numéricas complexas (vetores) que capturam o significado de palavras ou imagens.
- Target Encoding:** Transformar uma categoria em um número baseado na média da variável que queremos prever.

5. Seleção de Features

- VIF (Variance Inflation Factor):** Mede o quanto uma variável é "redundante" porque já é explicada por outras.
- RFE / SFS (Wrappers):** Métodos que testam várias combinações de variáveis, tirando e colocando uma a uma, para ver qual conjunto performa melhor.
- Permutation Importance:** Técnica que "bagunça" uma variável para ver o quanto o erro do modelo aumenta. Se o erro aumentar muito, aquela variável é importante.

Inspeção Geral dos Dados

Após compreender o problema, a próxima etapa é realizar uma inspeção geral dos dados brutos disponíveis.

Tamanho e Estrutura

Verifique o número de linhas e colunas, identifique os nomes das colunas e os tipos de dados que elas contêm (numéricos, categóricos, texto, etc.).

Origem e Processamento Prévio

Investigue de onde vêm os dados, como foram coletados e processados anteriormente, e se já foram utilizados para outros fins.

Qualidade dos Dados

Verifique se os exemplos fazem sentido contextualmente e se há anotações confiáveis em dados rotulados para treinamento supervisionado.

A inspeção manual de dados é frequentemente subestimada, mas possui um alto "valor-prestígio", pois mesmo uma breve observação pode revelar padrões e problemas que passariam despercebidos em análises automatizadas.

Higiene mínima (determinística)

Objetivo: Garantir a qualidade e a integridade do dataset, removendo ruídos e corrigindo erros para que as transformações subsequentes sejam precisas e confiáveis.

Esta etapa lida com a "higiene" dos dados e pode ser realizada antes de dividir o dataset, pois não envolve estatísticas que poderiam causar vazamento.

- **Identificação e Remoção de Duplicatas:**
 - **O que é:** Encontrar e excluir linhas idênticas no dataset.
 - **Por que fazer:** Dados duplicados podem enviesar o treinamento do modelo e superestimar a sua performance.
- **Padronização e Correção de Dados:**
 - **O que é:** Consertar erros de digitação, padronizar formatos e corrigir dados inconsistentes e corrompidos.
 - **Exemplo:** Unificar São Paulo, S. Paulo e SP para um único valor padronizado.
- **Não imputar nem remover outliers aqui.**



Split inicial (“Teste sagrado”)

Objetivo:

Criar um conjunto de TESTE intocado para medir o desempenho real.

Regra de ouro: o TESTE não decide nada (não influencia imputação, escala, seleção, tuning, nada).

Categoria	Limite Superior (Linhas)	Por que esse é o limite?
Pequeno	~5.000 a 10.000	Até aqui, técnicas como K-Fold são essenciais para não “perder” informação preciosa.
Médio	~500.000 a 1.000.000	É o limite do Excel e da Memória RAM de um PC comum. Acima disso, o Pandas (Python) começa a ficar lento.
Gigante	Infinito (Milhões+)	Quando os dados não cabem mais na RAM e você precisa de Processamento Distribuído ou Big Data.

* i.i.d. = independentes e identicamente distribuídos.



Checagem de esquema (regras fixas)”

Objetivo:

Validação de dados por regra, sem aprender nada das estatísticas do conjunto. Serve para barrar sujeira antes do pipeline (imputação/escala/etc.). É igual para treino/val/teste. Garantir consistência e coerência dos dados.

O que NÃO fazer aqui

Imputar, escalar, remover outliers, calcular limiares/médias ou recodificar com base em estatísticas.

Fronteira com limpeza mínima

Correções determinísticas (ex.: trim, trocar “Santa Catarina ” → “SC”, parse de número/data) pertencem à limpeza mínima.

Se não há regra fixa → reprova na checagem (sem “adivinhar” valor).

Exemplos rápidos

- UF “SC ” (com espaço) → limpeza mínima corrige.
- UF “SCC” → cheque de esquema reprova (quarentena).
- Idade 145 → cheque de esquema reprova (não clipa aqui).
- L = 105 → reprova (fora de [0,100]).



EDA (Análise Exploratória de Dados)

Objetivo:

A missão do EDA é atuar como a fase de diagnóstico do nosso dataset, gerando insights acionáveis sobre a qualidade, distribuição e relação das variáveis para que possamos tomar decisões informadas em cada passo do pré-processamento.

Este objetivo se desdobra em metas mais específicas:

- **Qualidade:** Entender a natureza e a extensão de problemas como valores ausentes, duplicatas e erros de digitação.
- **Distribuição:** Compreender a forma, a assimetria e a dispersão de cada variável, especialmente a do nosso alvo.
- **Relação:** Descobrir as correlações entre as variáveis e a sua relação com o alvo, para guiar a seleção e a engenharia de atributos.
- **Anomalias:** Identificar outliers e problemas de desbalanceamento de classes que precisam de tratamento específico.



EDA (Análise Exploratória de Dados)

Análise de Visão Geral (Initial Overview)

- **Dimensões do Dataset:** O número total de linhas e colunas.
- **Tipos de Dados:** Verificar se cada coluna tem o tipo de dado correto (numérico, categórico, booleano, data/hora).
- **Estatísticas Descritivas:** Gerar um resumo estatístico para todas as colunas numéricas (média, mediana, desvio padrão, mínimo, máximo, quartis). Isso dá uma visão rápida da distribuição dos dados.
- **Valores Ausentes:** Contar e exibir a porcentagem de valores ausentes por coluna. Isso informa quais colunas precisam de imputação e quão severo é o problema.
- **Dados Categóricos:** Contar o número de categorias únicas em cada coluna categórica. Isso ajuda a decidir a técnica de codificação (ex: One-Hot para poucas categorias, Target Encoding para alta cardinalidade).



EDA (Análise Exploratória de Dados)

Análise de Variáveis Individuais (Univariate Analysis)

- Para Variáveis Numéricas:

- **Histogramas e Gráficos de Densidade:** Visualizar a forma da distribuição dos dados. A distribuição é normal, assimétrica para a direita (right-skewed) ou para a esquerda (left-skewed)? Isso afeta a escolha da transformação.
- **Boxplots:** Identificar visualmente a presença de **outliers** e a dispersão dos dados.

- Para Variáveis Categóricas:

- **Gráficos de Barras (Bar Charts):** Visualizar a frequência de cada categoria. Isso é crucial para identificar classes desbalanceadas.



EDA (Análise Exploratória de Dados)

Análise de Relação entre Variáveis (Bivariate Analysis)

- **Correlação entre Variáveis Numéricas:**
 - **Matriz de Correlação (Heatmap):** Visualizar a correlação entre todas as variáveis numéricas. Isso ajuda a identificar a multicolinearidade (variáveis que se movem juntas), que pode ser um problema para modelos lineares.
- **Relação entre Atributos e a Variável Alvo (y):**
 - **Variável Numérica vs. Alvo (y) Numérica:** Gráficos de dispersão (scatter plots) para ver se há uma relação linear.
 - **Variável Numérica vs. Alvo (y) Categórica:** Boxplots ou gráficos de violino para ver como a distribuição da variável numérica varia entre as classes do alvo.
 - **Variável Categórica vs. Alvo (y) Numérica:** Gráficos de barras para ver como a média do alvo varia entre as categorias.



EDA (Análise Exploratória de Dados)

Sumário dos Problemas e Insights

Finalmente, o EDA deve culminar em uma lista de insights e ações a serem tomadas.

- **Lista de Outliers:** Quais colunas têm outliers e qual a estratégia para tratá-los (remover, transformar)?
- **Lista de Variáveis com Valores Ausentes:** Quais colunas precisam de imputação e qual método será usado (mediana, KNN)?
- **Lista de Atributos para Transformação:** Quais colunas precisam ser padronizadas ou normalizadas, e quais têm assimetria e podem se beneficiar de uma transformação logarítmica?
- **Identificação de Atributos para Engenharia:** Quais colunas podem ser combinadas ou transformadas para criar novas features (ex: extrair ano, mês de uma coluna de data).
- **Problema de Desbalanceamento de Classes:** Se o alvo for categórico, há um desbalanceamento? Isso exigirá um balanceamento posterior.



Pré-processamento: A Fase da Ação

Objetivo:

Transformar os dados brutos, usando as estatísticas e regras aprendidas no conjunto de treino, em um formato ideal para o algoritmo de Machine Learning (apenas no conjunto de treino).

Etapas Chave do Pré-processamento

1. Tratamento de Valores Ausentes

- **Problema:** Dados faltando (NaN) impedem a maioria dos modelos de funcionar.
- **Solução:** Preencher os valores ausentes com uma técnica adequada ao tipo de dado e à sua distribuição, como:
 - **Imputação por Mediana:** Para variáveis numéricas com outliers.
 - **Imputação por Média:** Para variáveis numéricas sem outliers significativos.
 - **Imputação por Moda:** Para variáveis categóricas.
 - **Imputação por Modelo (Ex: KNN):** Para casos complexos onde a relação entre as variáveis é importante.



Pré-processamento: A Fase da Ação

Etapas Chave do Pré-processamento

2. Tratamento de Variáveis Categóricas

- **Problema:** Modelos de ML não podem processar texto.
- **Solução:** Converter variáveis categóricas em um formato numérico.
 - **One-Hot Encoding:** Criar colunas binárias para cada categoria. Ideal para dados nominais (sem ordem) com poucas categorias.
 - **Label Encoding:** Atribuir um número inteiro a cada categoria. Ideal apenas para dados ordinais (com ordem natural).
 - **Outras Técnicas:** Mencionar o uso de Frequency Encoding ou Target Encoding para dados de alta cardinalidade.



Tipos comuns de dados

Dados Categóricos

- Eles representam qualidades ou características. Eles não possuem um valor numérico intrínseco que permita operações matemáticas como "somar" ou "tirar a média". Eles servem para classificar itens em grupos.

SKU	Produto	Cor	Quantidade	Preço
908721	Camisa	Azul	789	45,33
456552	Tênis	Vermelho	244	22,91
789921	Bermuda	Verde	387	25,92
872266	Camisa	Azul	154	17,56

Transformação de dados categóricos

Existem duas abordagens comuns para codificar dados categóricos:

- **O Ordinal Encoding** converte dados categóricos em códigos inteiros que variam de 0 a (número de categorias – 1).
 - **Quando usar:** Para variáveis categóricas que possuem uma ordem natural.
 - **Exemplos:**
 - Níveis de escolaridade (Ensino Fundamental, Médio, Superior).
 - Classificação de qualidade (Ruim, Regular, Bom, Excelente).
 - Tamanhos de roupas (P, M, G, GG).
 - **Vantagens:**
 - **Simplicidade e Eficiência:** É um método simples de implementar e usa apenas uma coluna para cada variável, economizando memória e processamento.
 - **Sem Aumento de Dimensionalidade:** Não cria novas colunas, o que é ideal para datasets com muitas categorias únicas.
 - **Desvantagens:**
 - **Cria Relação Falsa:** Se a variável não tiver uma ordem natural, o modelo pode interpretar a codificação numérica como uma relação de grandeza, como se "2" fosse o dobro de "1". Por exemplo, ao codificar cores (Azul=1, Verde=2, Vermelho=3), o modelo pode erroneamente assumir que a cor "Vermelho" é "maior" que a cor "Azul".

Transformação Ordinal Encoding

Quando usar?

Exemplo de Ordem Intrínseca (Quando USAR Ordinal Encoding)

Imagine que você está construindo um modelo para prever a nota de satisfação do cliente. Uma das suas variáveis é a qualidade do produto, com as seguintes categorias: **Ruim, Aceitável, Bom, Excelente**.

- **Ordem Real:** Existe uma ordem natural. Excelente é objetivamente melhor que Bom, que é melhor que Aceitável, e assim por diante.
- **Codificação:**
 - Ruim -> 0
 - Aceitável -> 1
 - Bom -> 2
 - Excelente -> 3

Por que isso funciona?

Ao atribuir esses números, você está ajudando o modelo. O algoritmo entenderá que um produto com qualidade 3 (Excelente) tem uma chance maior de resultar em uma nota de satisfação alta do que um produto com qualidade 0 (Ruim). O modelo pode usar essa relação numérica para aprender e fazer melhores previsões, pois a ordem numérica reflete a ordem real da variável.

Transformação de dados categóricos

- **One-Hot Encoding** transforma cada categoria nominal (sem ordem) em uma nova coluna binária (com 0s e 1s).
 - **Quando usar:** Para variáveis sem ordem intrínseca, como cores, cidades, ou tipos de produto.
 - **Exemplos:**
 - Para a categoria Cores com os valores Vermelho e Azul, o algoritmo cria duas novas colunas: Cor_Vermelho e Cor_Azul.
 - Um registro com a cor Vermelho terá 1 na coluna Cor_Vermelho e 0 na Cor_Azul.
 - **Vantagens:**
 - **Evita Falsa Ordem:** Impede que o modelo interprete uma hierarquia numérica inexistente entre as categorias.
 - **Compatibilidade:** Funciona bem com a maioria dos algoritmos de Machine Learning, especialmente os que usam distância (como K-Means e SVMs).
 - **Desvantagens:**
 - **Aumento de Dimensionalidade:** Cria muitas colunas novas, o que pode sobrecarregar o modelo, principalmente se a variável tiver muitas categorias.
 - **Colinearidade:** Pode criar dependência linear entre as colunas, o que pode ser um problema para alguns modelos (ex: Regressão Linear).

Transformação One hot encoding

SKU	Produto	Cor	Quantidade	Preço
908721	Camisa	Azul	789	45,33
456552	Tênis	Vermelho	244	22,91
789921	Bermuda	Verde	387	25,92
872266	Camisa	Azul	154	17,56



SKU	Bermuda	Camisa	Tênis	Vermelho	Verde	Azul	Quantidade	Preço
908721	0	1	0	0	0	1	789	45,33
456552	0	0	1	1	0	0	244	22,91
789921	1	0	0	0	1	0	387	25,92
872266	0	1	0	0	0	1	154	17,56

Pré-processamento: A Fase da Ação

Etapas Chave do Pré-processamento

3. Tratamento de Outliers e Assimetria

- **Problema:** Valores extremos (outliers) e distribuições assimétricas podem prejudicar o desempenho de modelos lineares e baseados em distância.
- **Solução:** Normalizar a distribuição dos dados numéricos.
 - **Transformação Logarítmica:** Para reduzir a assimetria acentuada e a influência de outliers.
 - **Winsorização:** Limitar os valores extremos a um certo percentil para mitigar o seu impacto.



Pré-processamento: A Fase da Ação

Etapas Chave do Pré-processamento

4. Escalonamento de Atributos Numéricos

- **Problema:** Variáveis com diferentes escalas (ex: Preço vs. Quartos) podem enviesar o modelo.
- **Solução:** Padronizar a escala de todas as variáveis numéricas para que contribuam igualmente para o modelo.
 - **Padronização (StandardScaler):** Escala os dados para ter média 0 e desvio padrão 1. Ideal para modelos que usam gradientes (Regressão Logística, SVM, Redes Neurais).
 - **Normalização (MinMaxScaler):** Escala os dados para um intervalo fixo (geralmente de 0 a 1). Ideal para modelos que usam distâncias (K-NN) e redes neurais que usam ativações como sigmoid.



Escalonamento de dados por padronização

A **padronização** redimensiona os dados para que eles tenham uma média de 0 e um desvio padrão de 1.

$$z = \frac{x - \mu}{\sigma}$$

Subtraímos a média (μ) de cada valor (x) e depois dividimos pelo desvio padrão (σ).

Escalonamento de dados por padronização

Suponha que tenhamos uma amostra que contém três pontos de dados com os seguintes valores: 50, 100, 150.

A média dos nossos dados seria 100, enquanto o desvio padrão da amostra seria 50.

$$(50 - 100)/50 = -50/50 = -1$$

$$(100 - 100)/50 = 0/50 = 0$$

$$(150 - 100)/50 = 50/50 = 1$$

O resultado da padronização é que nossa distribuição de dados agora tem uma média de 0 e um desvio padrão de 1

Escalonamento de dados por padronização

✓ Resultado (dados padronizados)

Idades originais:

csharp

 Copiar  Editar

[20, 22, 25, 30, 33]

Idades em z-score:

css

 Copiar  Editar

[-1.23, -0.82, -0.21, 0.82, 1.44]

🌟 Interpretação

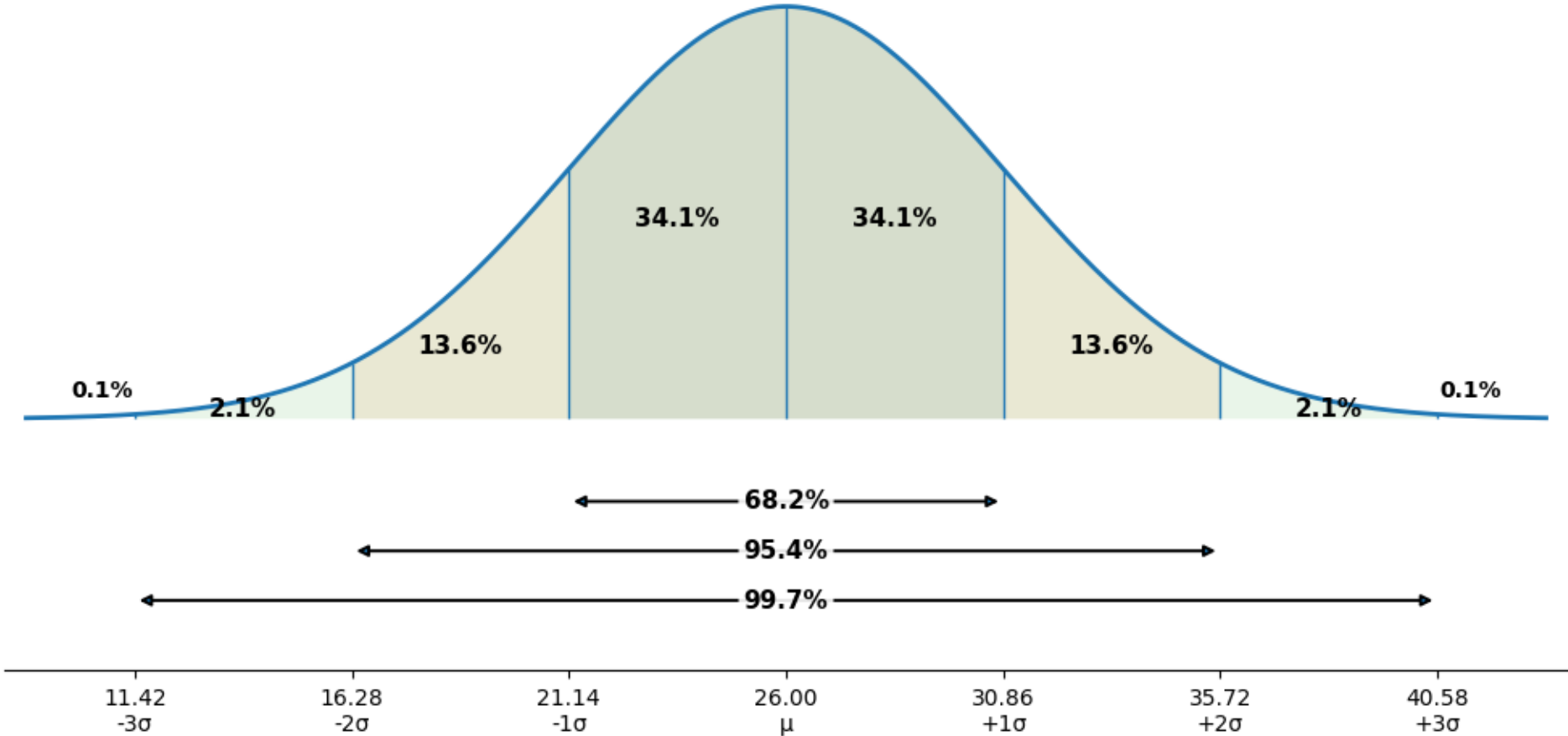
- O valor 25 ficou perto de 0 → está próximo da média.
- O valor 20 ficou em -1.23 → está 1.23 desvios abaixo da média.
- O valor 33 ficou em 1.44 → está 1.44 desvios acima da média.



Ou seja, o z-score mostra o quão distante um valor está da média em termos de desvio padrão.

Escalonamento de dados por padronização

Distribuição normal teórica com $\mu = 26$ e $\sigma = 4,86$



Curva normal ilustrativa baseada na média e no desvio padrão do exemplo.
Isso representa uma distribuição teórica, não os 5 pontos brutos isoladamente.

Escalonamento de dados por normalização

Suponha que tenhamos uma amostra que contém três pontos de dados com os seguintes valores: 50, 100, 150.

O valor mínimo (x_{min}) é 50, enquanto o valor máximo (x_{max}) é 150. O intervalo dos valores é $x_{max} - x_{min} = 150 - 50 = 100$.

$$(50 - 50)/100 = 0/100 = 0$$

$$(100 - 50)/100 = 50/100 = 0.5$$

$$(150 - 50)/100 = 100/100 = 1$$

O objetivo era redimensionar nossos dados em valores que variam de 0 a 1, e como você pode ver, foi exatamente isso que a fórmula fez.

Escalonamento de dados por normalização

3. Cálculos para cada valor

- Para $x = 20$:

$$x' = \frac{20 - 20}{13} = \frac{0}{13} = 0$$

- Para $x = 22$:

$$x' = \frac{22 - 20}{13} = \frac{2}{13} \approx 0.15$$

- Para $x = 25$:

$$x' = \frac{25 - 20}{13} = \frac{5}{13} \approx 0.38$$

- Para $x = 30$:

$$x' = \frac{30 - 20}{13} = \frac{10}{13} \approx 0.77$$

- Para $x = 33$:

$$x' = \frac{33 - 20}{13} = \frac{13}{13} = 1$$

✓ Resultado (dados normalizados com Min-Max)

Idades originais:

```
csharp
```

```
[20, 22, 25, 30, 33]
```

Idades normalizadas (0 a 1):

```
csharp
```

```
[0.00, 0.15, 0.38, 0.77, 1.00]
```

Quando usar

Casos em que não precisamos

1. Árvores de decisão, Random Forest, Gradient Boosting

- Árvores dividem por limiares, então escala não importa.
- Ex.: uma árvore pode separar “idade < 30” e “salário > 2000” sem precisar comparar as magnitudes.

2. Modelos que já lidam bem com escalas

- Alguns algoritmos probabilísticos (Naive Bayes, por exemplo).

3. Quando os dados já estão na mesma ordem de grandeza

- Se todas as variáveis já estão normalizadas por natureza (ex.: proporções, percentuais, índices entre 0 e 1).

Quando usar

Algoritmo / Modelo	Precisa normalizar?
KNN (K-Nearest Neighbors)	Sim
K-Means / Clustering	Sim
SVM (Support Vector Machine)	Sim
PCA (Análise de Componentes)	Sim
Redes Neurais (MLP, CNN, etc.)	Sim
Regressão Linear / Logística	Recomendado
Naive Bayes	Não
Árvore de Decisão	Não
Random Forest	Não
Gradient Boosting (XGBoost, LightGBM, CatBoost)	Não

Extração/Criação de Features

Objetivo:

Transformar os dados brutos, através do conhecimento do problema, em variáveis mais significativas para o modelo. Criar novas "pistas" que o algoritmo não conseguiria encontrar sozinho.

Principais Estratégias

- **Transformações de Tempo**
 - Desdobrar datas (como "2024-08-17") em atributos relevantes como ano, mês, dia da semana e hora.
 - Isso permite que o modelo identifique e aprenda com padrões cíclicos (ex: picos de vendas em meses específicos ou dias da semana).
- **Criação de Interações**
 - Combinar duas ou mais variáveis para criar uma nova que tenha maior significado para o modelo.
 - Exemplo: Criar o Preço_por_m2 dividindo o Preço de um imóvel pela Área_m2. Essa variável única pode ser mais preditiva do que as duas originais.



Seleção de Atributos

Objetivo:

Remover variáveis irrelevantes ou redundantes para focar o modelo nos atributos mais importantes. Isso melhora a performance, reduz o risco de overfitting e acelera o treinamento

Principais Estratégias

- **Métodos de Filtro:** Usar estatísticas simples (ex: correlação com a variável alvo) para classificar e selecionar as melhores features.
- **Métodos de Wrapper:** Usar o próprio modelo para avaliar a performance de diferentes combinações de features (ex: Eliminação Recursiva de Atributos - RFE).
- **Métodos Embutidos:** Algoritmos que fazem a seleção de atributos automaticamente durante o treinamento (ex: Lasso Regression).



O Passo a Passo Correto

1.DIVIDIR: Primeiro de tudo, você separa os dados (Treino, Validação, Teste).

2.PREPARAR O TREINO (`fit + transform`): * Você olha para o Treino e decide: "Vou usar a média X para preencher buracos e a escala Y".

1. Você aplica essas mudanças no Treino. Agora o Treino está "limpo".

3.PREPARAR VALIDAÇÃO E TESTE (`transform apenas`):

1. **IMPORTANTE:** Você prepara eles **AGORA**, antes de rodar o modelo.

2. Você usa as regras (X e Y) que tirou do Treino e aplica na Validação e no Teste.

3. *Por que agora?* Porque se você tentar validar o modelo depois, mas os dados de validação ainda estiverem "sujos" ou em escalas diferentes, o resultado da validação será lixo.

4.TREINAR O MODELO:

1. Agora sim, você entrega o **Treino já limpo** para o algoritmo aprender.

5.VALIDAR / TESTAR:

1. Você entrega a **Validação já limpa** para o modelo fazer as previsões e você ver se ele está bom.





Anexos

Análise Estatística dos Dados

Média (μ ou $\bar{x}$): Medida de tendência central que representa o valor médio de um conjunto de dados. É calculada somando todos os valores e dividindo pelo número de observações.

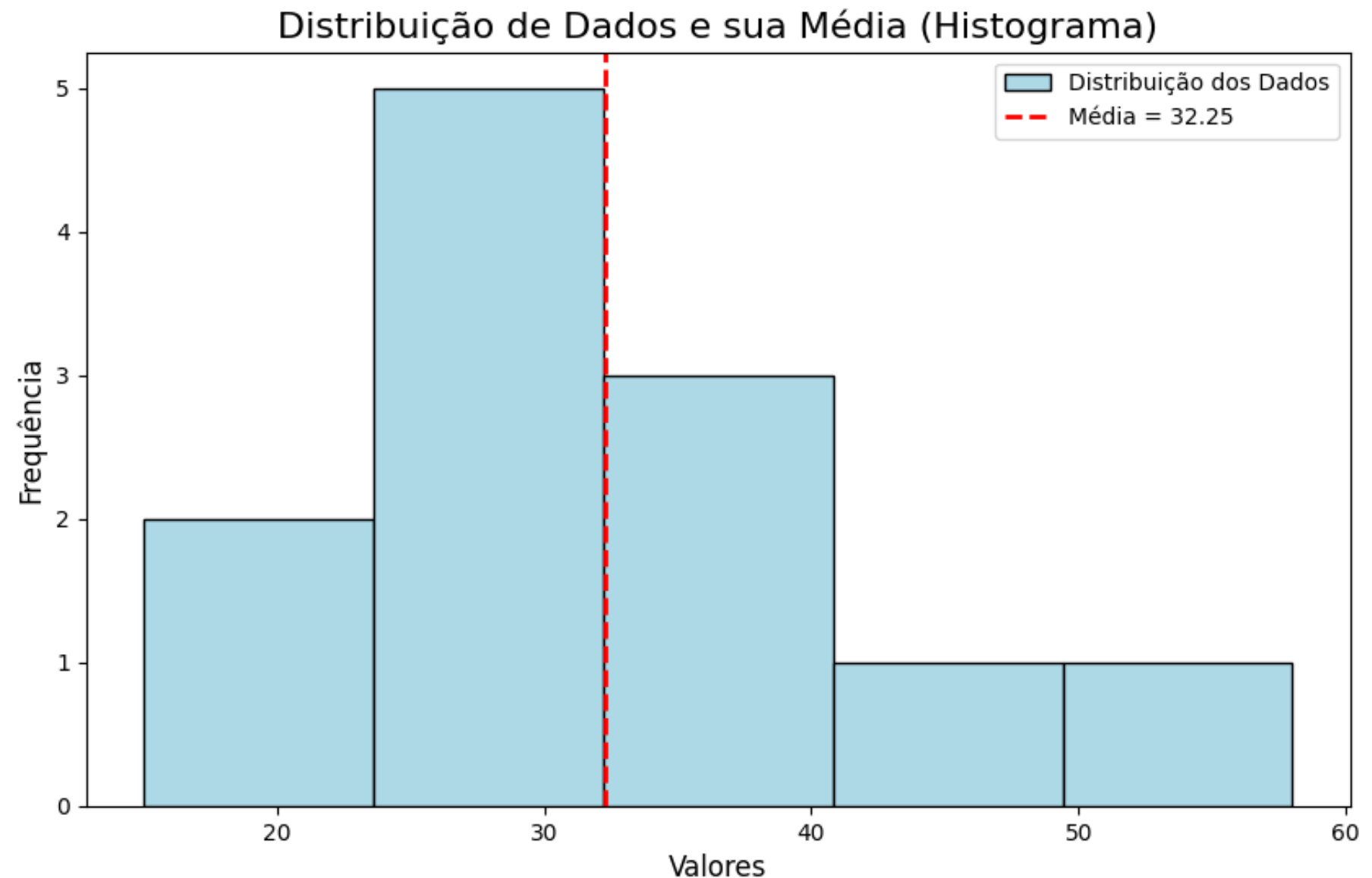
$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Descrição dos Parâmetros:

- **$\bar{x}$ (x-barra):** Representa a **média aritmética** do conjunto de dados. É o valor central que resume o conjunto de observações.
- **n :** Indica o **número total de observações** ou elementos no conjunto.
- **Σ (Sigma):** É o símbolo de **somatório**. Ele indica que todos os valores de x_i devem ser somados, começando do primeiro ($i=1$) até o último (n).
- **$i=1$:** Esta parte do somatório indica o **início do intervalo** da soma. A variável i é o índice que começa em 1, representando o primeiro elemento do conjunto de dados.
- **x_i :** Refere-se a cada **valor individual** ou observação no conjunto de dados. O índice i varia de 1 a n , representando cada um dos valores a serem somados ($x_1, x_2, x_3, \dots, x_n$).

<> Python

```
def calcular_media(numeros):
    """
```

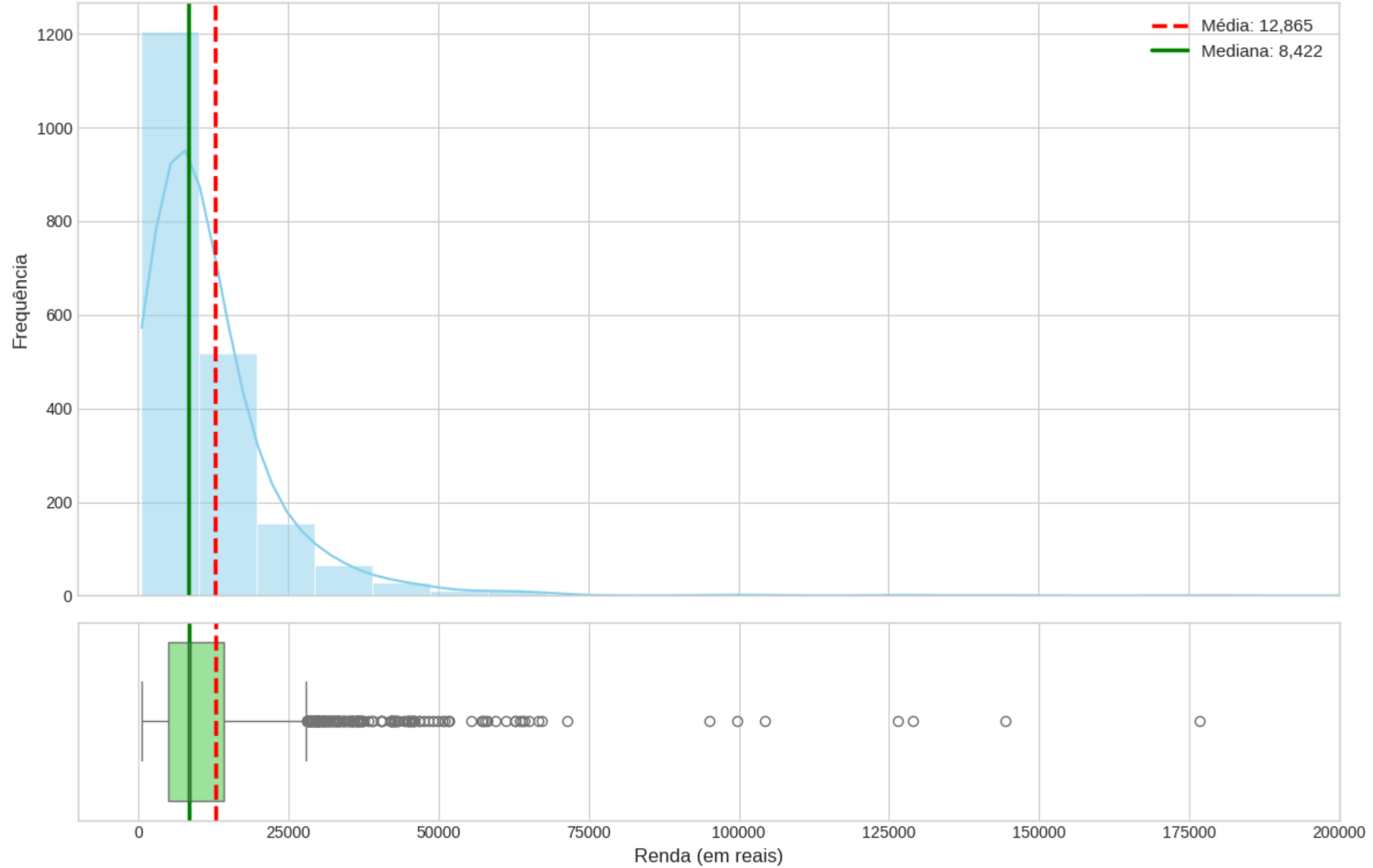


Análise Estatística dos Dados

```
def calcular_mediana(numeros):  
    """  
    Calcula a mediana de uma lista de números.  
    A mediana é o valor do meio de um conjunto de dados ordenado.  
  
    Retorna a mediana ou None se a lista estiver vazia.  
    """  
    if not numeros: # Verifica se a lista é vazia  
        return None  
  
    # 1. Ordena a lista de números  
    # A mediana exige que os dados estejam em ordem.  
    lista_ordenada = sorted(numeros)  
  
    n = len(lista_ordenada)  
  
    # 2. Verifica se a quantidade de elementos é ímpar ou par  
    # A operação % 2 (módulo) retorna 0 para números pares e 1 para ímpares.  
    if n % 2 == 1:  
        # Se n for ímpar, a mediana é o elemento do meio.  
        # A divisão inteira // 2 nos dá o índice exato.  
        indice_central = n // 2  
        return lista_ordenada[indice_central]  
    else:  
        # Se n for par, a mediana é a média dos dois elementos do meio.  
        indice_1 = n // 2 - 1  
        indice_2 = n // 2  
  
        elemento_1 = lista_ordenada[indice_1]  
        elemento_2 = lista_ordenada[indice_2]  
  
        return (elemento_1 + elemento_2) / 2.0
```

```
# --- Programa Principal ---  
  
# Definir listas para testar ambos os casos (ímpar e par)  
lista_impar = [10, 20, 30, 40, 50]  
lista_par = [10, 20, 30, 40]  
lista_com_outlier = [10, 20, 30, 40, 500000] # Para demonstrar a robustez  
  
# Testar o caso ímpar  
mediana_impar = calcular_mediana(lista_impar)  
if mediana_impar is not None:  
    print(f"A mediana da lista {lista_impar} é: {mediana_impar}")  
else:  
    print("Não foi possível calcular a mediana, pois a lista está vazia.")  
  
# Testar o caso par  
mediana_par = calcular_mediana(lista_par)  
if mediana_par is not None:  
    print(f"A mediana da lista {lista_par} é: {mediana_par}")  
else:  
    print("Não foi possível calcular a mediana, pois a lista está vazia.")  
  
# Testar o caso com outlier  
mediana_outlier = calcular_mediana(lista_com_outlier)  
if mediana_outlier is not None:  
    print(f"A mediana da lista com outlier é: {mediana_outlier}")  
else:  
    print("Não foi possível calcular a mediana, pois a lista está vazia.")
```

Distribuição de Renda com Outliers



Análise Estatística dos Dados

```
from collections import Counter

def calcular_moda(numeros):
    """
    Calcula a moda (ou modas) de uma lista de números ou strings.
    A moda é o valor mais frequente. Um conjunto de dados pode ter uma ou mais modas.

    Retorna uma lista contendo a(s) moda(s).
    Retorna uma lista vazia se não houver moda (todos os elementos são únicos).
    """
    if not numeros: # Verifica se a lista está vazia
        return []

    # Usa o Counter para contar a frequência de cada elemento na lista
    frequencias = Counter(numeros)

    # Encontra a frequência máxima (o número de ocorrências do elemento mais comum)
    max_frequencia = max(frequencias.values())

    # Encontra todos os elementos que têm a frequência máxima
    modas = [chave for chave, valor in frequencias.items() if valor == max_frequencia]

    # Se todos os elementos têm a mesma frequência (geralmente 1), não há moda única.
    # Ex: [1, 2, 3] -> max_frequencia = 1. Todos os elementos são a moda.
    # O retorno de uma lista vazia neste caso é uma convenção comum.
    if len(modas) == len(numeros):
        return []
```

```
# --- Programa Principal ---

# 1. Conjunto de dados unimodal (uma moda)
lista_unimodal = [1, 2, 2, 3, 4, 5, 5, 5, 6, 7]
moda_unimodal = calcular_moda(lista_unimodal)
print(f"A moda da lista {lista_unimodal} é: {moda_unimodal}")

# 2. Conjunto de dados bimodal (duas modas)
lista_bimodal = [10, 20, 20, 30, 40, 40, 50]
moda_bimodal = calcular_moda(lista_bimodal)
print(f"A moda da lista {lista_bimodal} é: {moda_bimodal}")

# 3. Conjunto de dados sem moda (todos os elementos únicos)
lista_sem_moda = [100, 200, 300, 400, 500]
moda_sem_moda = calcular_moda(lista_sem_moda)
if moda_sem_moda:
    print(f"A moda da lista {lista_sem_moda} é: {moda_sem_moda}")
else:
    print(f"A lista {lista_sem_moda} não tem uma moda única.")
```

Análise Estatística dos Dados

A variância e o desvio padrão são ferramentas essenciais no pré-processamento de machine learning, principalmente para uma técnica chamada **padronização (Standardization)**.

O Problema Resolvido: Escalas Diferentes

Muitos algoritmos de machine learning (como SVMs, K-Means, Redes Neurais e métodos baseados em gradiente) calculam distâncias entre pontos ou se baseiam em gradientes. Quando suas features (colunas) têm escalas muito diferentes, o algoritmo pode ser enganado.

Exemplo: Imagine um conjunto de dados com duas features: Área_m2 (valores de 50 a 500) e Número_Quartos (valores de 1 a 5). A distância entre duas casas de 100 m² e 150 m² é de 50. A distância entre 3 e 4 quartos é de 1. O algoritmo vai dar muito mais peso à Área_m2 simplesmente porque seus valores são numericamente maiores. Ele pode ignorar a importância de Número_Quartos.

Análise Estatística dos Dados

1. Análise Exploratória de Dados (AED)

A AED é o primeiro e mais importante passo em qualquer projeto de ciência de dados, visando obter intuição e compreensão valiosas do projeto através da sumarização e visualização dos dados.

Estimativas de Localização:

- **Média: O que é:** A soma de todos os valores, dividida pelo número de valores.
O que faz: Fornece um "valor típico" ou a tendência central dos dados.
- **Mediana: O que é:** O valor que ocupa a posição central em uma lista de dados classificada.
O que faz: É uma estimativa robusta de localização, menos sensível a valores extremos (outliers) do que a média.
- **Média Aparada: O que é:** A média calculada após a exclusão de um número fixo de valores extremos de cada ponta dos dados.
O que faz: Elimina a influência de outliers, sendo uma alternativa robusta à média comum.
- **Média Ponderada: O que é:** Calcula a média multiplicando cada valor por um peso e dividindo pela soma dos pesos.
O que faz: Usada quando alguns valores são intrinsecamente mais variáveis ou quando os dados coletados não representam igualmente os grupos de interesse.

Análise Estatística dos Dados

Estimativas de Variabilidade:

- **Desvio-padrão e Variância:** **O que é:** Medidas de dispersão baseadas nas diferenças quadráticas dos valores em relação à média. O desvio-padrão é a raiz quadrada da variância. **O que faz:** Quantificam o quão espalhados os dados estão em torno do valor central. São sensíveis a outliers.
- **Desvio Absoluto Mediano da Mediana (MAD):** **O que é:** A mediana do valor absoluto dos desvios da mediana. **O que faz:** É uma estimativa robusta de variabilidade, não influenciada por valores extremos.
- **Percentis e Amplitude Interquartil (IQR):** **O que é:** Percentis indicam um valor abaixo do qual uma dada porcentagem dos dados cai. A IQR é a diferença entre o 75º e o 25º percentil. **O que faz:** Oferecem uma abordagem robusta para estimar a dispersão, observando a distribuição dos dados classificados, sendo valiosos para resumir as "caudas" (extremos) da distribuição.

Análise Estatística dos Dados

Visualização da Distribuição de Dados:

- **Boxplots: O que é:** Um gráfico que resume a distribuição de dados usando quartis. **O que faz:** Visualiza rapidamente a mediana, a amplitude interquartil e a presença de outliers, sendo útil para comparar distribuições lado a lado.
- **Tabelas de Frequências e Histogramas: O que é:** A tabela de frequências divide a gama de valores em segmentos e conta quantos valores caem em cada um. O histograma é a representação gráfica dessa tabela. **O que faz:** Ambos fornecem uma visão da forma da distribuição dos dados, incluindo a presença de assimetrias ou múltiplos picos.
- **Gráficos de Densidade: O que é:** Uma versão suavizada do histograma que mostra a distribuição dos valores dos dados como uma linha contínua, geralmente calculada por estimativa de densidade de Kernel. **O que faz:** Oferece uma representação clara da distribuição, especialmente útil para comparar distribuições.
- **Gráficos de Barras: O que é:** Ferramenta visual para exibir uma única variável categórica. **O que faz:** Representa a frequência ou proporção de cada categoria em barras, facilitando a comparação visual entre elas.

Análise Estatística dos Dados

Correlação e Relações entre Variáveis:

- **Coeficiente de Correlação e Matriz de Correlação: O que é:** O coeficiente mede a associação linear entre duas variáveis numéricas (de -1 a $+1$). A matriz de correlação exibe esses coeficientes para múltiplos pares de variáveis. **O que faz:** Ajuda a identificar se variáveis se movem juntas (correlação positiva), em direções opostas (negativa) ou não têm relação linear (zero), sendo útil para entender relacionamentos e identificar redundâncias entre preditoras.
- **Gráficos de Dispersão: O que é:** Um gráfico com uma variável no eixo X e outra no eixo Y, onde cada ponto é um registro. **O que faz:** Permite a visualização direta do relacionamento entre duas variáveis, identificando padrões, grupos ou outliers.
- **Tabelas de Contingência: O que é:** Uma tabela que registra as contagens entre duas ou mais variáveis categóricas. **O que faz:** Útil para resumir e analisar a relação entre variáveis categóricas, mostrando, por exemplo, a distribuição de empréstimos por nota e status.
- **Compartimentação Hexagonal e Contornos: O que é:** Gráficos que visualizam a densidade de pontos para duas variáveis numéricas em grandes conjuntos de dados. **O que faz:** Agrupam registros em hexágonos coloridos ou contornos, revelando padrões de densidade que seriam ocultos por gráficos de dispersão muito densos.

Análise Estatística dos Dados

Correlação e Relações entre Variáveis:

- **Gráficos Violino: O que é:** Uma variação do boxplot que incorpora uma estimativa de densidade. **O que faz:** Permite ver nuances na distribuição de uma variável numérica condicionada por categorias de outra variável, combinando a informação da dispersão dos boxplots com a forma da densidade.
- **Facetas (Condicionamento): O que é:** Um conceito gráfico que estende a análise visual a mais de duas variáveis, criando múltiplos subgráficos baseados nas categorias de uma variável condicionante. **O que faz:** Permite comparar padrões ou distribuições entre diferentes subgrupos nos dados.